An End-to-End Guide to Containerization and Orchestration with ECS & ECR

This project demonstrates a real-world DevOps workflow: taking a local Node.js application, containerizing it with **Docker**, and deploying it to a highly available, serverless environment using **Amazon ECS (Elastic Container Service)** and **AWS Fargate**.

📚 Educational Guide: The Tech Stack

Before building, it is important to understand the components of a modern cloud architecture:

1. Docker (The Container)

- **What it is:** A tool that packages code and all its dependencies so the application runs quickly and reliably from one computing environment to another.
- **Why use it:** It eliminates the "it works on my machine" problem.
- **Key Concept:** A **Docker Image** is a read-only template with instructions for creating a Docker container.

2. Amazon ECR (The Registry)

- **What it is:** A fully managed Docker container registry that makes it easy for developers to store, manage, and deploy Docker container images.
- **Why use it:** It provides a secure and scalable location to host your images before they are pulled by ECS.

3. Amazon ECS & AWS Fargate (The Orchestrator)

- **ECS:** A highly scalable, high-performance container management service.
- **Fargate:** A serverless compute engine for containers that works with ECS. It removes the need to provision and manage servers, allowing you to pay only for the resources used.
- **Concepts:** * **Cluster:** A logical grouping of tasks or services.
  - **Task Definition:** A blueprint that describes how a docker container should launch (CPU, memory, ports, etc.).
  - **Task:** An instantiation of a Task Definition within a cluster.

4. IAM (Security)

- **What it is:** Identity and Access Management manages permissions for AWS services.
- **Role in this project:** We use an **ECS Task Execution Role** to allow ECS to pull images from ECR and send logs to CloudWatch.

🛠️ Implementation Steps

Phase 1: Build Server Setup (EC2)

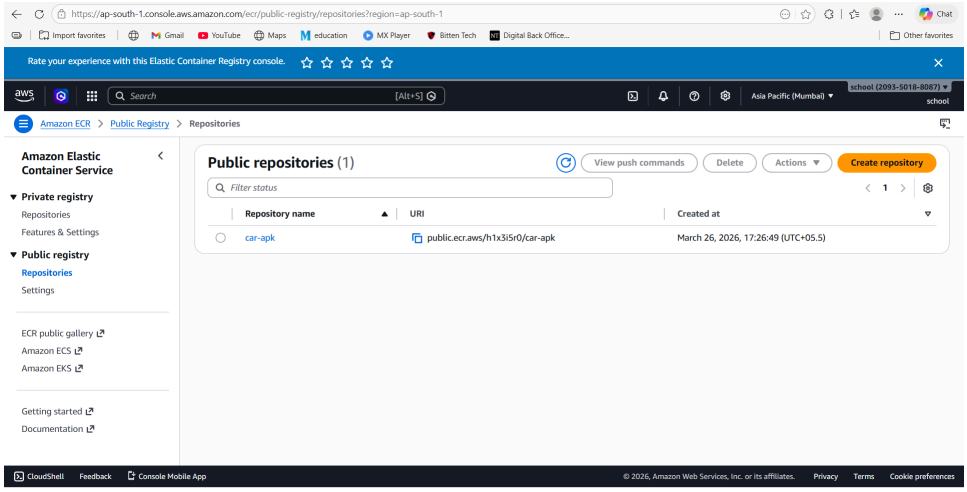
We use an Ubuntu VM as our "Build Server" to prepare our image.

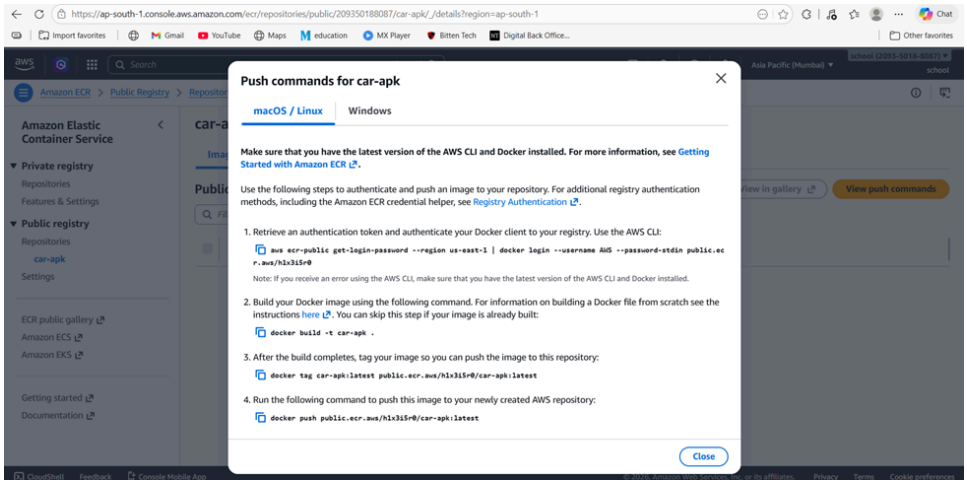
1. **Launch Instance:** Create an Ubuntu VM named ECS-vm.
2. **Clone Repository:** clone the car showroom application code to the VM.
3. **Install Docker:** Initialize the container engine and add the user to the docker group to run commands without sudo .
4. **Install AWS CLI:** Configure the command line interface to interact with your AWS account

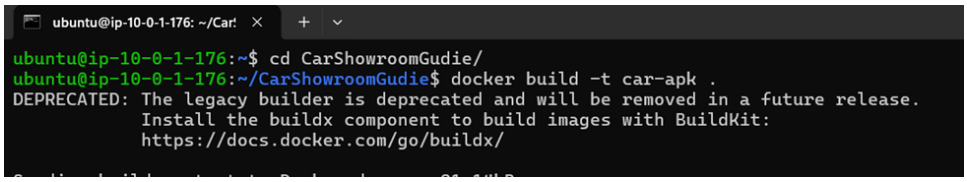
---

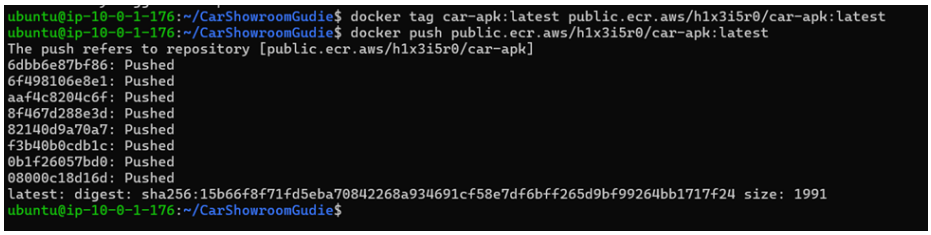
Phase 2: Dockerizing and Pushing to ECR

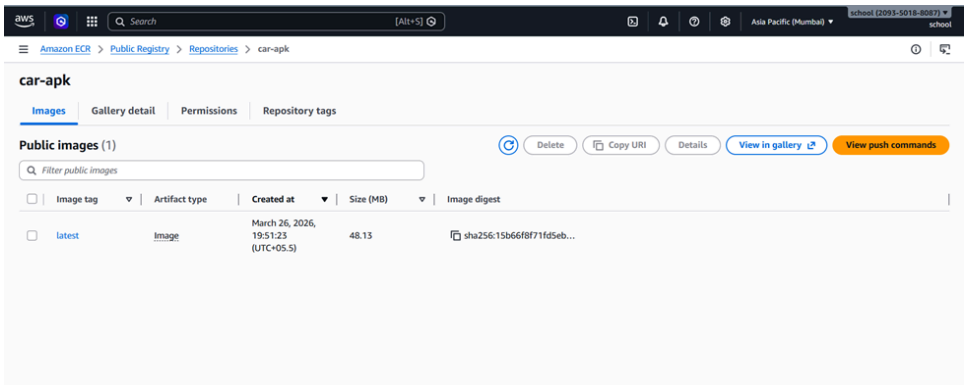
1. **Create Repository:** Create a public ECR repository named car-apk.
2. **Authenticate:** Use the get-login-password command to authenticate your local Docker client to the ECR registry.
3. **Build Image:** Run docker build -t car-apk . inside your project directory.
4. **Tag & Push:** Tag the image with your ECR URI and push it to the cloud .









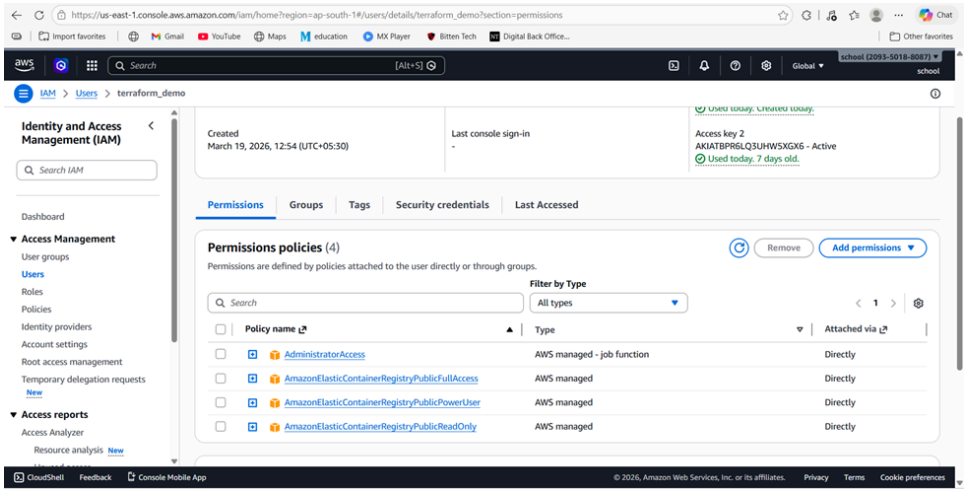


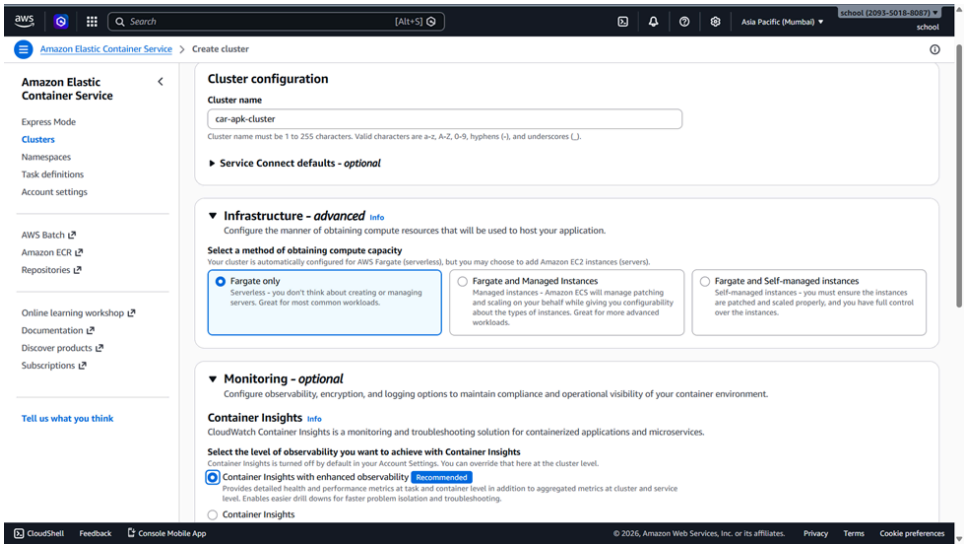
Phase 3: ECS Configuration

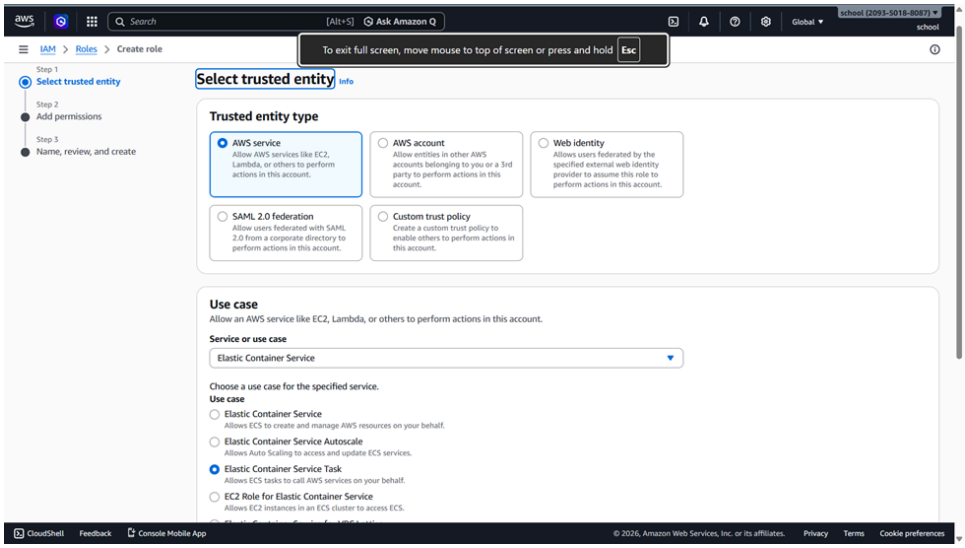
1. **Create Cluster:** Use the ECS console to create a cluster (e.g., carApkcluster) using **Fargate**.

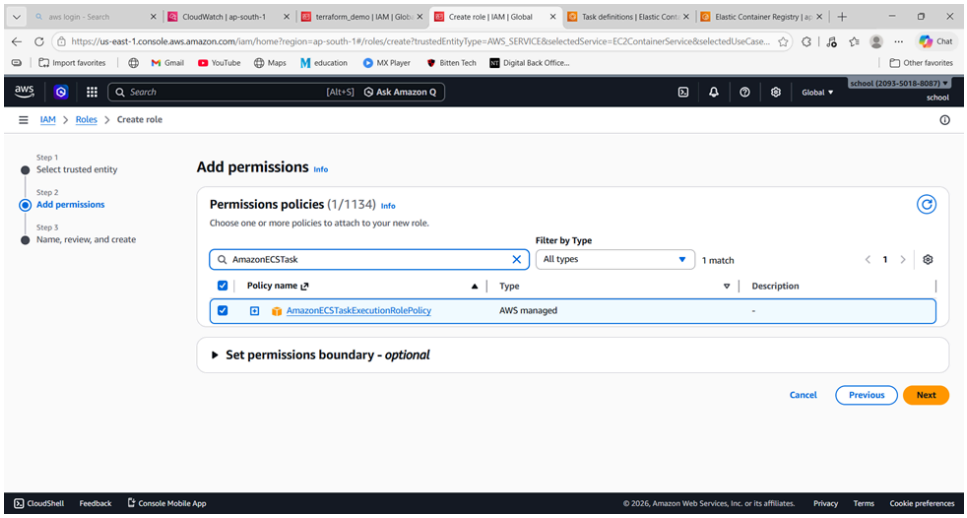
2. **IAM Role:** Create a role with AmazonECSTaskExecutionRolePolicy so the service has permission to run your container.

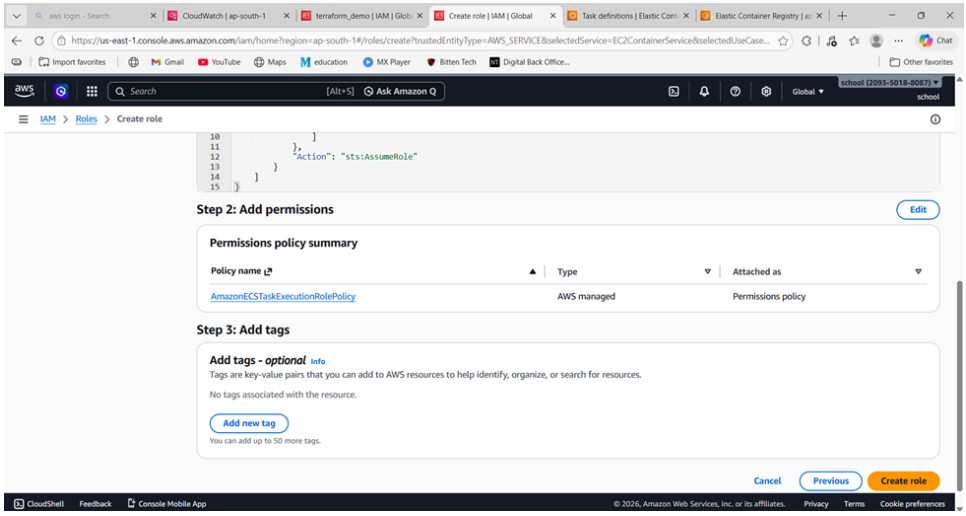
3. **Task Definition:** Create a new definition (e.g., carAPkTaskdefine) specifying :
    - **Launch Type:** Fargate.
    - **OS/Architecture:** Linux/X86_64.
    - **Task Size:** 1 vCPU and 3 GB Memory .
    - **Container Port:** 8000 (standard for this Node.js app).






IAM > Roles > Create role
To exit full screen, move mouse to top of screen or press and hold Esc
Step 1
Select trusted entity
Step 2
Add permissions
Step 3
Name, review, and create
Select trusted entity Info
Trusted entity type
AWS service
Allow AWS services like EC2, Lambda, or others to perform actions in this account.
AWS account
Allow entities in other AWS accounts belonging to you or a 3rd party to perform actions in this account.
Web identity
Allows users federated by the specified external web identity provider to assume this role to perform actions in this account.
SAML 2.0 federation
Allow users federated with SAML 2.0 from a corporate directory to perform actions in this account.
Custom trust policy
Create a custom trust policy to enable others to perform actions in this account.
Use case
Allow an AWS service like EC2, Lambda, or others to perform actions in this account.
Service or use case
Elastic Container Service
Choose a use case for the specified service.
Use case
Elastic Container Service
Allows ECS to create and manage AWS resources on your behalf.
Elastic Container Service Autoscale
Allows Auto Scaling to access and update ECS services.
Elastic Container Service Task
Allows ECS tasks to call AWS services on your behalf.
EC2 Role for Elastic Container Service
Allows EC2 instances in an ECS cluster to access ECS.
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


IAM > Roles > Create role
Step 1
Select trusted entity
Step 2
Add permissions
Step 3
Name, review, and create
Add permissions Info
Permissions policies (1/1134) Info
Choose one or more policies to attach to your new role.
Filter by Type
AmazonECSTask
All types
1 match
Policy name
Type
Description
AmazonECSTaskExecutionRolePolicy
AWS managed
-
Set permissions boundary - optional
Cancel
Previous
Next
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

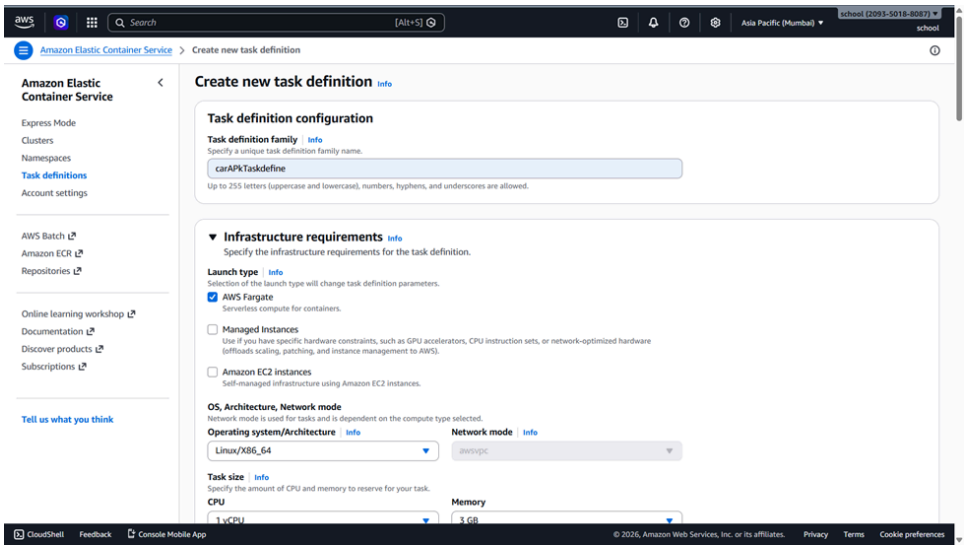

IAM > Roles > Create role
"Action": "sts:AssumeRole"
Step 2: Add permissions
Edit
Permissions policy summary
Policy name
Type
Attached as
AmazonECSTaskExecutionRolePolicy
AWS managed
Permissions policy
Step 3: Add tags
Add tags - optional Info
Tags are key-value pairs that you can add to AWS resources to help identify, organize, or search for resources.
No tags associated with the resource.
Add new tag
You can add up to 50 more tags.
Cancel
Previous
Create role
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

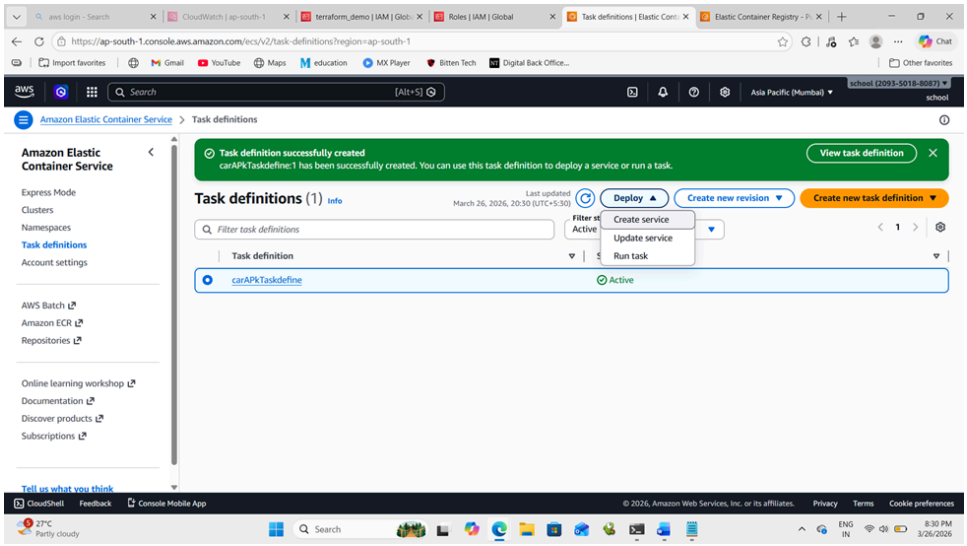
Phase 4: Running and Troubleshooting

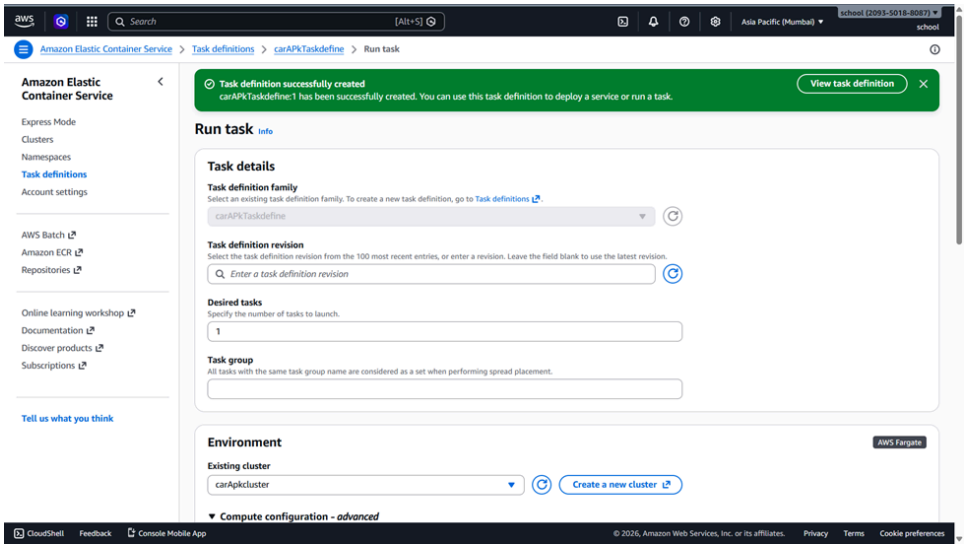
1. **Run Task:** Launch the task within your cluster.

2. **Access App:** Find the **Public IP** in the Task Configuration and paste it into your browser.

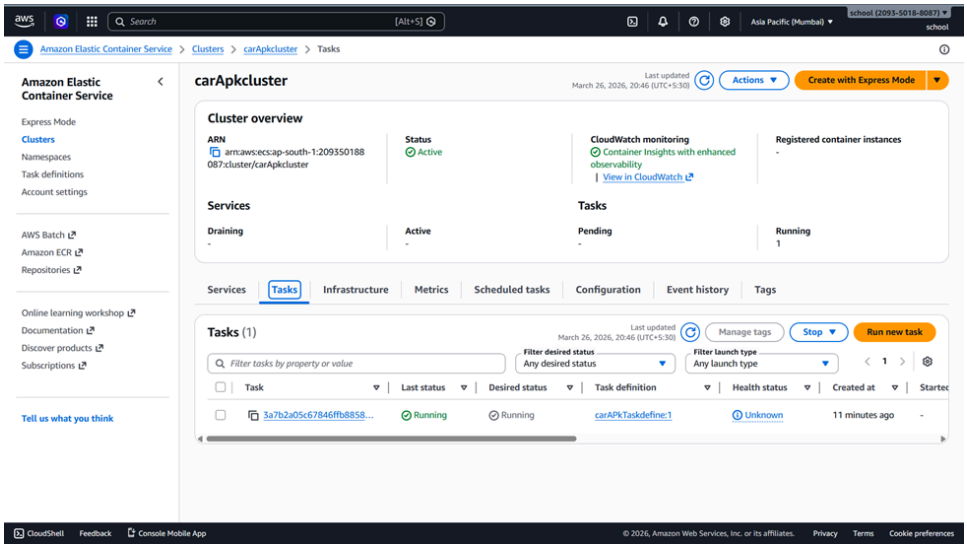
3. **Security Group Fix:** If the page won't load, go to the **Security Group Inbound Rules** and add a rule for **Port 8000** allowing traffic from "Anywhere" (0.0.0.0/0) .

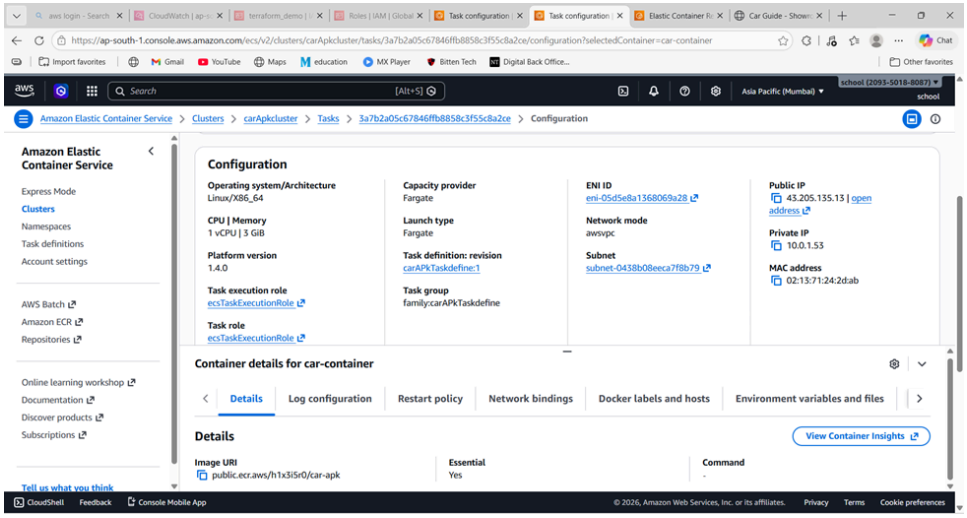






Go to the task click one task id and copy the ip address and run on the broswer

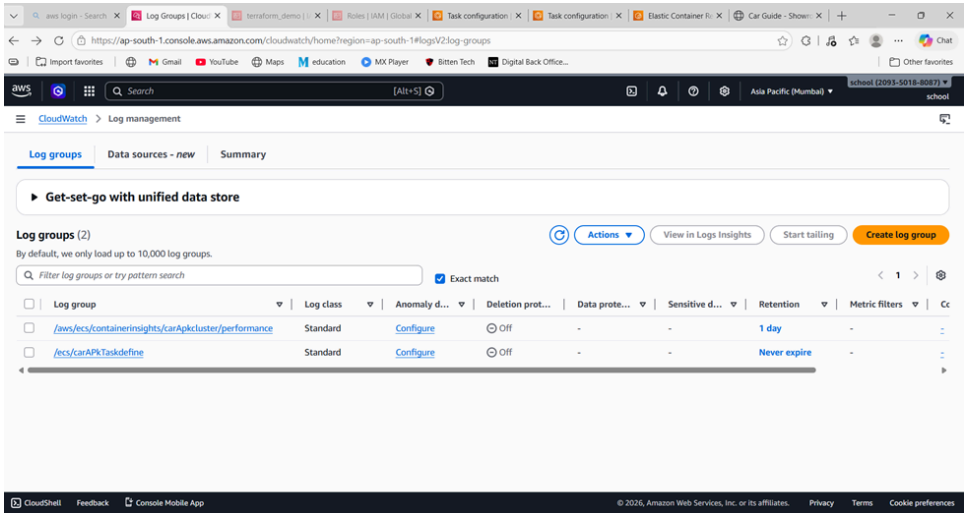


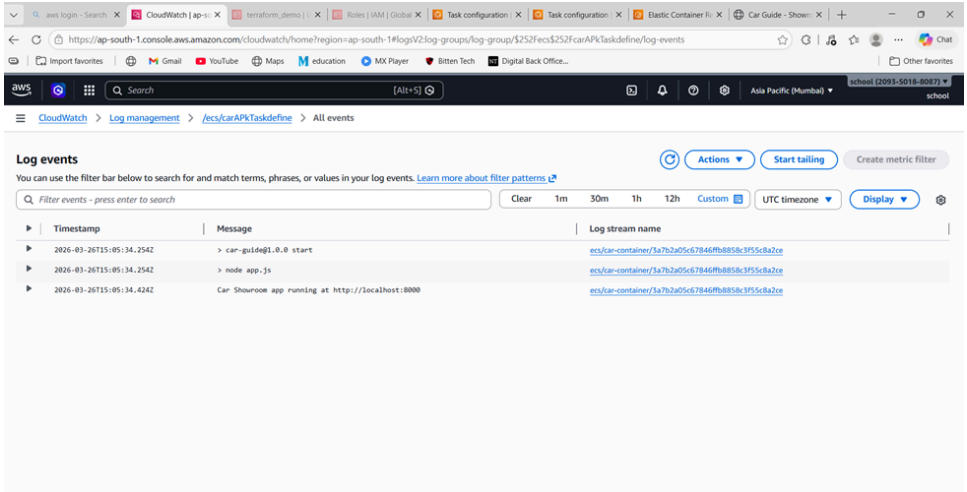


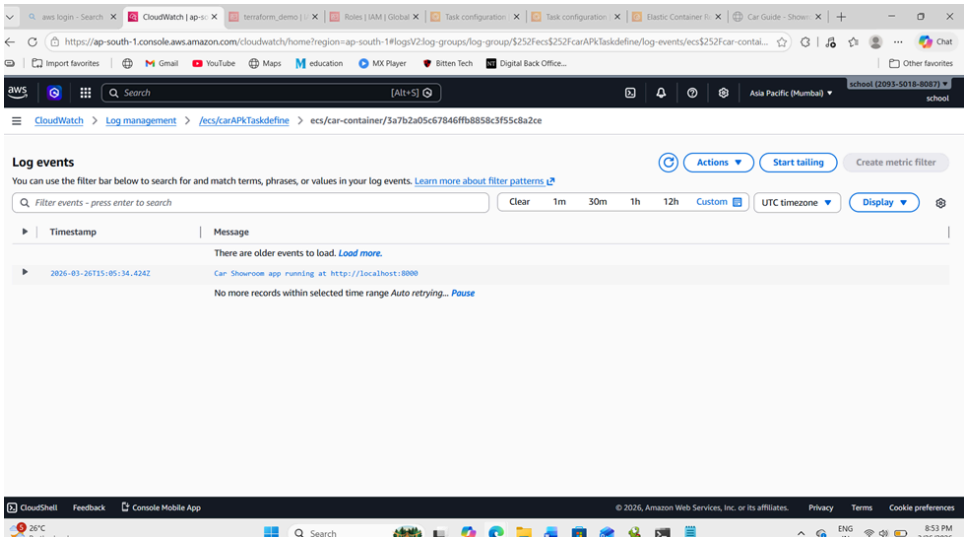
📊 Monitoring with CloudWatch

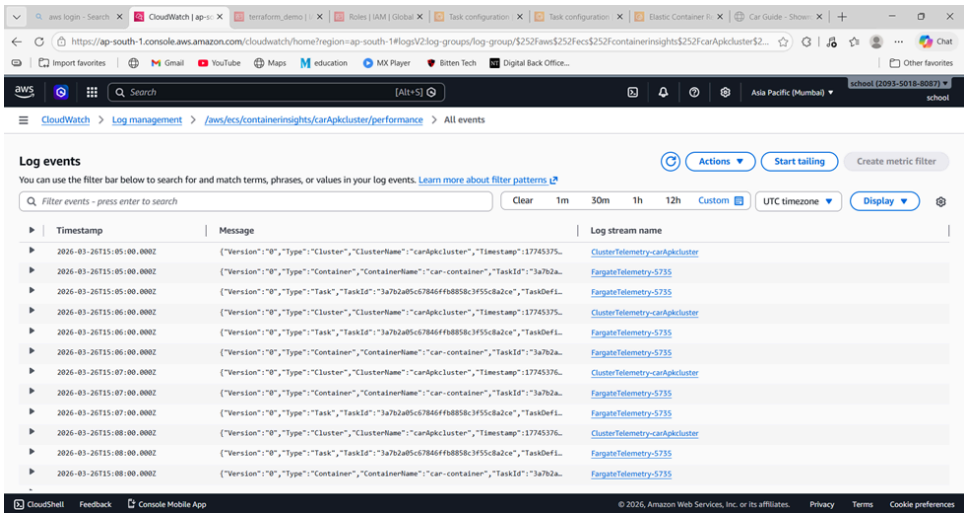
All container logs are sent to **AWS CloudWatch**. This is essential for:

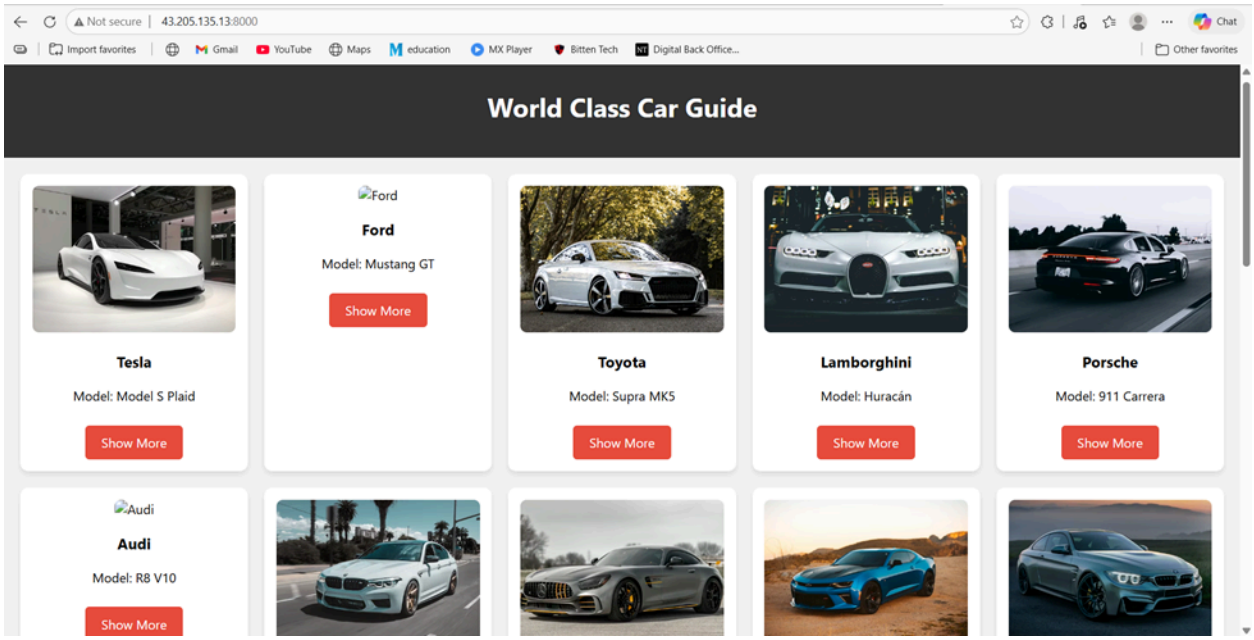
- **Debugging:** Seeing application errors in real-time.
- **Performance:** Monitoring CPU and Memory utilization.




https://ap-south-1.console.aws.amazon.com/cloudwatch/home?region=ap-south-1#logsV2:log-groups/log-group/$252Fecs$252FcarAPkTaskdefine/log-events
CloudWatch > Log management > /ecs/carAPkTaskdefine > All events
Log events
You can use the filter bar below to search for and match terms, phrases, or values in your log events. Learn more about filter patterns
Actions
Start tailing
Create metric filter
Filter events - press enter to search
Clear
1m
30m
1h
12h
Custom
UTC timezone
Display
Timestamp
Message
Log stream name
2026-03-26T15:05:34.254Z
> car-guide@1.0.0 start
ecs/car-container/3a7b2a05c67846ffb8858c3f55c8a2ce
2026-03-26T15:05:34.254Z
> node app.js
ecs/car-container/3a7b2a05c67846ffb8858c3f55c8a2ce
2026-03-26T15:05:34.424Z
Car Showroom app running at http://localhost:8000
ecs/car-container/3a7b2a05c67846ffb8858c3f55c8a2ce


https://ap-south-1.console.aws.amazon.com/cloudwatch/home?region=ap-south-1#logsV2:log-groups/log-group/$252Fecs$252FcarAPkTaskdefine/log-events/ecs$252Fcar-contai...
CloudWatch > Log management > /ecs/carAPkTaskdefine > ecs/car-container/3a7b2a05c67846ffb8858c3f55c8a2ce
Log events
You can use the filter bar below to search for and match terms, phrases, or values in your log events. Learn more about filter patterns
Actions
Start tailing
Create metric filter
Filter events - press enter to search
Clear
1m
30m
1h
12h
Custom
UTC timezone
Display
Timestamp
Message
There are older events to load. Load more.
2026-03-26T15:05:34.424Z
Car Showroom app running at http://localhost:8000
No more records within selected time range Auto retrying... Pause


https://ap-south-1.console.aws.amazon.com/cloudwatch/home?region=ap-south-1#logsV2:log-groups/log-group/$252Faws$252Fecs$252Fcontainerinsights$252FcarApkcluster$2...
CloudWatch > Log management > /aws/ecs/containerinsights/carApkcluster/performance > All events
Log events
You can use the filter bar below to search for and match terms, phrases, or values in your log events. Learn more about filter patterns
Actions
Start tailing
Create metric filter
Filter events - press enter to search
Clear
1m
30m
1h
12h
Custom
UTC timezone
Display
Timestamp
Message
Log stream name
2026-03-26T15:05:00.000Z
{"Version":"0","Type":"Cluster","ClusterName":"carApkcluster","Timestamp":17745375...
ClusterTelemetry-carApkcluster
2026-03-26T15:05:00.000Z
{"Version":"0","Type":"Container","ContainerName":"car-container","TaskId":"3a7b2a...
FargateTelemetry-5735
2026-03-26T15:05:00.000Z
{"Version":"0","Type":"Task","TaskId":"3a7b2a05c67846ffb8858c3f55c8a2ce","TaskDefi...
FargateTelemetry-5735
2026-03-26T15:06:00.000Z
{"Version":"0","Type":"Cluster","ClusterName":"carApkcluster","Timestamp":17745375...
ClusterTelemetry-carApkcluster
2026-03-26T15:06:00.000Z
{"Version":"0","Type":"Task","TaskId":"3a7b2a05c67846ffb8858c3f55c8a2ce","TaskDefi...
FargateTelemetry-5735
2026-03-26T15:06:00.000Z
{"Version":"0","Type":"Container","ContainerName":"car-container","TaskId":"3a7b2a...
FargateTelemetry-5735
2026-03-26T15:07:00.000Z
{"Version":"0","Type":"Cluster","ClusterName":"carApkcluster","Timestamp":17745376...
ClusterTelemetry-carApkcluster
2026-03-26T15:07:00.000Z
{"Version":"0","Type":"Container","ContainerName":"car-container","TaskId":"3a7b2a...
FargateTelemetry-5735
2026-03-26T15:07:00.000Z
{"Version":"0","Type":"Task","TaskId":"3a7b2a05c67846ffb8858c3f55c8a2ce","TaskDefi...
FargateTelemetry-5735
2026-03-26T15:08:00.000Z
{"Version":"0","Type":"Cluster","ClusterName":"carApkcluster","Timestamp":17745376...
ClusterTelemetry-carApkcluster
2026-03-26T15:08:00.000Z
{"Version":"0","Type":"Task","TaskId":"3a7b2a05c67846ffb8858c3f55c8a2ce","TaskDefi...
FargateTelemetry-5735
2026-03-26T15:08:00.000Z
{"Version":"0","Type":"Container","ContainerName":"car-container","TaskId":"3a7b2a...
FargateTelemetry-5735



📝 Conclusion

This project demonstrates how to leverage serverless technologies to host applications without managing infrastructure. By using **ECS Fargate**, we ensure that our car showroom application is scalable and cost-effective.